DAY 2 · TERRAFORM ACROSS CLOUDS

One Terraform codebase for 3 clouds? Terraform says no.

~/infra terraform init

```
# "one module for every cloud"
module "network" {
source = "./modules/${var.cloud}"
}

| Error: Variables not allowed
|
| Variables may not be used here.
```

FAILS

WHAT & WHY

What Terraform gives a multi-cloud team, and what it doesn't

Terraform is an infrastructure-as-code tool: you declare resources in HCL, and **providers** translate them into each cloud's API calls. OpenTofu is the open-source fork with the same language and workflow.

It gives you

One language · one workflow (plan → apply) · a reviewable plan before every change · state that tracks what exists · one CI pipeline for every cloud

It doesn't give you

A universal "network" or "VM" resource · portable configs · identical behavior · protection from bad module design

The core idea: reuse the **workflow and the interfaces**. Accept that the **resources** are cloud-specific.

HOW IT WORKS

The Terraform workflow, step by step

01 **init:** downloads providers and modules, configures the state backend, and writes `.terraform.lock.hcl` with provider checksums

02 **validate:** checks syntax and internal consistency without calling any cloud API

03 **plan:** refreshes state, builds the dependency graph, and shows exactly what will be created, changed or destroyed

04 **apply:** executes the plan in dependency order (parallel where possible) and records the result in state

05 **state:** the record mapping your config to real resource IDs. Remote, locked and encrypted in any team setting

Why the plan matters: it's your change review. Saving it (`-out=tfplan`) guarantees apply does exactly what was reviewed.

REFERENCE ARCHITECTURE

Shared modules, one root per cloud and environment

```
infra/
├── modules/                          # reusable building blocks
│   └── network/
│       ├── aws/      main.tf  variables.tf  outputs.tf
│       ├── azure/    main.tf  variables.tf  outputs.tf
│       └── gcp/      main.tf  variables.tf  outputs.tf
└── live/                             # root configs: each has its own state
    ├── aws/
    │   ├── prod/network/          backend.tf  main.tf
    │   └── dev/network/
    ├── azure/prod/network/
    └── gcp/prod/network/
```

→ **modules/** define *what* a network is; **live/** decides *where* and with which values

→ Small roots = small state = fast plans and a small blast radius

→ Same variable names and outputs in every cloud's module

THE PROCESS

Building a multi-cloud Terraform codebase

01 **Pin versions:** Terraform and every provider, with the lock file committed

02 **Remote state per root:** S3 / Azure Blob / GCS, locked and encrypted

03 **Design the module interface first:** inputs, validation and outputs

04 **Implement it per cloud:** thin modules, no cross-cloud conditionals

05 **Compose roots:** one per cloud × environment × component

06 **Automate checks in CI:** fmt, validate, lint, security scan, plan

07 **Apply only after review:** from CI, with OIDC credentials

08 **Version modules:** tag releases so changes roll out deliberately

STEP 1 · PIN PROVIDERS

Three providers, one versions file

```hcl
terraform {
  required_version = "~> 1.9"
  required_providers {
    aws     = { source = "hashicorp/aws",     version = "~> 6.0" }
    azurerm = { source = "hashicorp/azurerm", version = "~> 4.0" }
    google  = { source = "hashicorp/google",  version = "~> 6.0" }
  }
}

provider "aws"     { region = "us-east-1" }
provider "azurerm" { features {} }          # required block
provider "google" {
  project = "acme-prod"
  region  = "us-central1"
}
```

→ **Pin to each provider's current major version** (the values above are examples) and upgrade on purpose

→ **Commit .terraform.lock.hcl** so every run uses the same provider builds

→ **In practice, a root usually configures one cloud's provider;** shown together here for reference

STEP 3 · THE INTERFACE

Design the module contract first

```
# modules/network/<cloud>/variables.tf (same in every cloud)
variable "name" {
  type        = string
  description = "Network name prefix"
}
variable "env" {
  type = string
  validation {
    condition     = contains(["dev", "staging", "prod"], var.env)
    error_message = "env must be dev, staging or prod."
  }
}
variable "cidr" {
  type = string
  validation {
    condition     = can(cidrhost(var.cidr, 0))
    error_message = "cidr must be a valid CIDR block."
  }
}
variable "tags" {
  type    = map(string)
  default = {}
}
# outputs.tf: every cloud returns network_id and subnet_ids
```

STEP 4 · IMPLEMENTATIONS

Same contract, cloud-specific resources

```
# aws/main.tf
resource "aws_vpc" "this" {
  cidr_block = var.cidr
  tags       = merge(var.tags, { Name = "${var.name}-${var.env}" })
}
# azure/main.tf (Azure-only inputs: location, resource group)
resource "azurerm_virtual_network" "this" {
  name                = "${var.name}-${var.env}"
  address_space       = [var.cidr]
  location            = var.location
  resource_group_name = var.resource_group_name
  tags                = var.tags
}
# gcp/main.tf (the CIDR lives on the subnet, not the VPC)
resource "google_compute_network" "this" {
  name                    = "${var.name}-${var.env}"
  auto_create_subnetworks = false
}
resource "google_compute_subnetwork" "this" {
  name          = "${var.name}-${var.env}"
  network       = google_compute_network.this.id
  ip_cidr_range = var.cidr
  region        = var.region
}
```

STEP 5 · COMPOSE ROOTS

Choosing the cloud: folders, not variables

```
# ✗ Fails: a module source must be a literal string
module "network" { source = "../modules/network/${var.cloud}" }

# ✓ live/aws/prod/network/main.tf
module "network" {
  source = "git::https://github.com/acme/infra-modules.git//network/aws?
ref=v1.4.0"
  name   = "payments"
  env    = "prod"
  cidr   = "10.10.0.0/16"
}

# Multi-region in one cloud: provider aliases
provider "aws" {
  alias  = "eu"
  region = "eu-west-1"
}
module "network_eu" {
  source    = "../../../../modules/network/aws"
  providers = { aws = aws.eu }
  name      = "payments"
  env       = "prod"
  cidr      = "10.11.0.0/16"
}
```

STEPS 6–7 · ORCHESTRATION IN CI

Every change goes through the same gates

```
# On every pull request
terraform fmt -check -recursive    # consistent formatting
terraform init -input=false
terraform validate                 # syntax and references
tflint --recursive                 # provider-aware linting
trivy config .                     # security misconfigurations
terraform plan -out=tfplan -input=false
terraform show -no-color tfplan    # post the plan for review

# After approval, on merge
terraform apply -input=false tfplan
```

→ **CI authenticates with OIDC** to AWS, Azure and GCP: no stored cloud keys

→ **One job per root**, so a change to Azure never touches AWS state

→ **Reviewers read the plan**, not just the code diff

PITFALLS

8 mistakes in multi-cloud Terraform

- → **The universal module:** count = var.cloud == "aws" ? 1 : 0 everywhere
- → **Copy-paste per cloud:** three folders that drift in naming and validation
- → **Unpinned providers:** a new major version changes behavior overnight
- → **Lock file not committed:** different builds on laptops and in CI
- → **One giant state:** slow plans and a huge blast radius
- → **Applying from laptops:** no review, no audit trail, stale state
- → **Unversioned shared modules:** one edit hits every environment at once
- → **Hard-coded values:** regions, IDs and CIDRs buried inside modules

NEED-TO-KNOWS

What every Terraform professional should know

→ A module **source can't contain variables**; choose clouds by folder

→ **Provider aliases** deploy the same module to several regions or accounts

→ **for_each** beats count when items can be added or removed (stable addresses)

→ **moved** blocks rename resources without destroying them; **import** blocks adopt existing ones

→ **lifecycle prevent_destroy** protects critical resources like databases

→ **Workspaces** share one config; separate directories are clearer for prod vs dev

→ **Sensitive outputs** are hidden in logs, but still stored in state

→ **tflint** catches provider-specific errors that validate can't

CHECKLIST

Multi-cloud Terraform checklist

- ☐ Terraform and providers pinned; lock file committed
- ☐ Remote, locked, encrypted state for every root
- ☐ One module interface per capability, identical across clouds
- ☐ Input validation on shared variables
- ☐ One root per cloud × environment × component
- ☐ fmt, validate, tflint and a security scan on every PR
- ☐ Plan reviewed; apply from CI with OIDC only
- ☐ Shared modules versioned with Git tags
- ☐ for_each for collections; moved/import for refactors
- ☐ prevent_destroy on stateful resources

HANDS-ON LAB

Try it: Build one module contract, two clouds

01 Create `modules/network/aws` and `modules/network/gcp` with identical variables.tf and outputs.tf

02 Implement each with its own resources (aws_vpc vs google_compute_network + subnetwork)

03 Run the checks above; fix anything tflint reports

04 Optional: plan both from a small live/ root in a sandbox account

```
terraform -chdir=modules/network/aws init -backend=false
terraform -chdir=modules/network/aws validate
terraform -chdir=modules/network/gcp init -backend=false
terraform -chdir=modules/network/gcp validate
terraform fmt -check -recursive
tflint --recursive
```

Teardown: Validate and lint need no cloud resources. If you ran plan/apply in a sandbox, run terraform destroy.

FURTHER READING

Go deeper: official references

HashiCorp: Modules and module sources
developer.hashicorp.com/terraform/language/modules

HashiCorp: Terraform style guide
developer.hashicorp.com/terraform/language/style

Google Cloud: Best practices for using Terraform
cloud.google.com/docs/terraform/best-practices-for-terraform

AWS Prescriptive Guidance: Terraform AWS provider best practices
docs.aws.amazon.com/prescriptive-guidance/latest/terraform-aws-provider-best-practices/

TFLint and Trivy (linting and security scanning)
github.com/terraform-linters/tflint · aquasecurity.github.io/trivy

Docs change. Always check the current version before you design.

YOUR TURN

Would you use one repository or separate repositories for each cloud?

Share your setup in the comments. I read and reply to every one.

Follow Deji Ajishe

Day 2 of my DevOps series

♻️ Repost to help an engineer on your team

linkedin.com/in/deji-ajishe-64a35722a